

Cpp_homewrok 보고서

22013451 최건우

1.

기본 클래스를 정의하고 이를 상속받아 여러 도형 클래스를 구현하는 과정은 마치 다양한 모양의 레고 블록을 조립하는 것처럼 흥미로웠습니다. 특히, CPoint 클래스 변수를 통해 각 도형의 특성을 쉽게 정의할 수 있다는 점이 매우 유용하게 느껴졌습니다. 커브 도형을 위해 vector를 사용해 점들의 집합을 관리하는 부분은 처음에는 조금 복잡하게 느껴졌지만, 구현해보니 그 필요성을 이해할 수 있었습니다. 도형을 표현하는 기본 클래스 CMyShape를 정의하고, 이를 상속받아 사각형(CMyRect), 원(CMyCircle), 커브(CMyCurve), 별(CMyStar) 클래스를 각각 구현했습니다. 사각형을 그리기 위해 필요한 두 개의 CPoint 클래스 변수를 추가했고, 원과 별의 중점을 나타내는 CPoint 변수 한 개와 크기를 나타내는 int형 변수를 CMyShape의 멤버 변수로 설정하여 자식 클래스에서 쉽게 사용할 수 있도록 했습니다. 커브는 점들을 연결하는 방식으로 그려지므로, 점들의 집합을 저장하기 위해 vector 헤더 파일을 포함시키고, CMyShape 클래스에 vector<CPoint> 멤버 변수를 추가하여 점들의 집합을 관리할 수 있도록 했습니다.

2.

int형 변수를 통해 도형을 구분하는 방식은 간단하면서도 매우 효율적이어서, 이러한 작은 설계 요소들이 프로그램의 유연성과 기능성을 크게 향상시킨다는 것을 배울 수 있었습니다. 이러한 작업을 통해 객체 지향 프로그래밍의 유용성을 직접 체험할 수 있었고, 배운 내용을 실습하며 자신감을 얻었습니다. 각 도형이 그려질 수 있도록 CMyShape 클래스에 draw 함수를 가상 함수로 선언하고, 각 도형 클래스마다 이를 재정의했습니다. CChildview 클래스에 vector<CMyShape*>형 멤버 변수를 추가하여 도형들이 이벤트 발생 시마다 추가되도록 했습니다. 왼쪽 마우스 버튼을 누르면 해당 도형을 동적으로 할당하고, 마우스 버튼을 누른 채 드래그하면 도형의 크기를 조절할 수 있게 했습니다. 메뉴와 툴바에서 각 도형을 선택할 수 있도록 만들었으며, CChildView 클래스에 int형 변수를 추가해 도형을 구분했습니다. 이 변수는 0이면 사각형, 1이면 원, 2이면 커브, 3이면 별을 나타내며, 각 도형에 대한 메뉴나 툴바의 이벤트 처리는 이 변수를 통해 해당 도형에 맞게 설정되도록 했습니다.

3.

화면이 깜박거리는 문제를 해결하기 위해 더블 버퍼링을 설정하고 EraseBkgnd 이벤트를 추가하는 과정은 처음에는 조금 복잡하게 느껴졌습니다. 하지만 실제로 구현해보니, 화면이 매끄럽게 그려지는 것을 보며 성취감을 느꼈습니다. 화면이 깜박거리는 것을 방지하기 위해 더블 버퍼링을 설정하고, 백그라운드를

지웠다가 다시 그리는 방식으로 처리했습니다. 이를 위해 EraseBkgnd 이벤트를 추가하여, 무언가를 그리는 작업 중에도 화면이 깜박거리지 않도록 했습니다.

4.

Select 가상 함수를 활용해 각 도형을 선택할 수 있게 만들고, 이를 통해 다양한 도형을 효과적으로 관리할 수 있다는 점이 매우 흥미로웠습니다. 별의 영역을 확인하는 과정에서 Ray Casting Algorithm을 적용할 때는 약간의 혼란과 함께 새로운 알고리즘을 배우는 즐거움을 느꼈습니다. 도형을 클릭하여 선택할 수 있는 기능을 구현하기 위해, 그리기 기능과 선택 기능을 변수로 분리했습니다. 마우스 왼쪽 버튼을 눌렀을 때 마우스 위치가 도형의 영역 안에 있다면 도형이 선택되도록, CMyShape 클래스에 CPoint 변수를 인자로 받는 Select 가상 함수를 선언하고 각 도형 클래스에서 이를 재정의했습니다. 별의 영역을 확인하는 것이 어려웠기 때문에, 구글링을 통해 점이 다각형 내부에 있는지 확인하는 Ray Casting Algorithm을 적용했습니다. 이 알고리즘은 점에 대한 수평선을 그어 다각형의 선분과 교차하는 횟수를 세어, 그 횟수가 홀수이면 점이 다각형 내부에 있다는 원리입니다. 이를 통해 별의 영역을 확인할 수 있었습니다. 각 도형이 선택되었는지 확인하기 위해 CMyShape에 bool형 변수 sel을 선언하여, 점이 도형 안에 있다면 sel을 true로, 그렇지 않다면 false로 설정했습니다. 또한, shift 키를 사용해 여러 도형을 선택할 수 있도록, shift 키가 눌렸는지 확인할 push_shift 변수를 만들었습니다. KeyDown 이벤트에서 shift 키를 누르면 push_shift를 true로, KeyUp 이벤트에서 shift 키를 놓으면 push_shift를 false로 설정했습니다. push_shift가 true일 때는 이전에 선택된 도형들을 유지하여 여러 도형을 동시에 선택할 수 있도록 했습니다. 선택된 도형은 도형을 둘러싼 사각형 점선이 보이도록 하기 위해 selectRect 가상 함수를 선언하고 각 도형마다 이를 재정의했습니다. 도형을 둘러싼 사각형을 그리기 위해서는 도형의 가장 작은 x값과 가장 큰 x값, 가장 작은 y값과 가장 큰 y값을 알아야 했기 때문에, returnX 함수와 returnY 함수를 가상 함수로 선언하고 각 도형마다 이를 재정의했습니다. returnX와 returnY 함수는 인자로 false를 받을 때 도형의 x좌표 중 가장 큰 값과 y좌표 중 가장 큰 값을 반환하고, true를 받을 때는 x좌표 중 가장 작은 값과 y좌표 중 가장 작은 값을 반환하도록 했습니다.

5.

PointShape 변수를 통해 도형이 선택되었는지 여부를 확인하고, PointSelect 함수를 작성하는 동안 논리적인 사고를 요구하는 부분이 많았기 때문에 약간의 어려움이 있었습니다. 하지만 PointSelect 함수와 returnX, returnY 함수들을 활용하여 드래그한 영역 안에 도형이 포함되는지 확인하는 과정을 거치면서, 점점 자신감이 생겼습니다.

빈 공간에서 드래킹하여 선택 영역을 그리기 위해, CChildView 클래스 내에 도형이 선택되었는지 확인하는 bool형 변수 PointShape를 만들었습니다. 이 변수를 설정하기 위해 CMyShape 클래스에 bool값을 반환하는 PointSelect

함수를 추가했습니다. PointSelect 함수는 Select 함수와 유사하지만, 마우스 포인터가 위치한 곳에 도형이 있으면 false 를, 도형이 없으면 true 를 반환하도록 정의했습니다. 따라서 PointShape 가 false 일 때 드래그하면 선택 영역이 표시되도록 했습니다. 드래그한 영역 안에 도형이 있는지 확인하기 위해 returnX 함수와 returnY 함수를 사용하여 가장 큰 x 값과 y 값, 가장 작은 x 값과 y 값을 찾았습니다. 이 네 개의 좌표가 드래그한 영역 안에 모두 포함되어 있으면, 해당 도형은 선택된 것으로 간주합니다.

6.

PointShape 변수를 사용해 조건을 확인하고, 이동 기능을 구현하는 ShapeMove 가상 함수를 만드는 작업은 논리적인 사고를 많이 필요로 했습니다. 각 도형 클래스에서 ShapeMove 함수를 재정의하여 실제로 도형이 움직이도록 했을 때, 마우스를 움직이는 대로 도형이 따라오는 것을 보며 큰 성취감을 느꼈습니다. 도형을 이동시킬 때는 PointShape 변수가 true인지 확인하여, true일 경우에만 도형을 이동시키도록 설계했습니다. 도형 이동을 구현하기 위해 마우스의 위치 변화량을 x와 y 변화량으로 나누고, 이 두 변수를 인자로 받는 ShapeMove 가상 함수를 만들었습니다. 이후, 각 도형 클래스에서 이 함수를 재정의하여 도형이 실제로 움직이도록 구현했습니다.

7.

Delete 키 이벤트를 통해 도형을 삭제하고 메모리를 해제하는 작업은 프로그램의 안정성과 성능에 직접적인 영향을 미치기 때문에 신중하게 접근해야 했습니다. Delete 키가 눌렸다는 KeyDown 이벤트가 발생하면 선택된 도형을 삭제하고, 동적 할당된 메모리를 해제하도록 구현했습니다.

8.

GroupShape 클래스를 통해 여러 도형을 하나의 그룹으로 묶는 작업을 하면서 객체 지향 프로그래밍의 강력함을 실감했습니다. 도형이 그룹에 속해 있는지 여부를 확인하고, 그룹 내 그룹 문제를 해결하는 과정에서 많은 생각을 하게 되었습니다. gr 변수를 통해 도형이 어느 그룹에 속해 있는지 관리하고, Grouped 함수를 통해 이 문제를 해결했을 때는 큰 성취감을 느꼈습니다. 도형을 그룹화하기 위해 도형들을 관리할 수 있는 GroupShape 클래스를 만들었습니다. 이 클래스는 CMyShape*를 벡터로 관리하여 여러 도형을 그룹에 추가할 수 있도록 했습니다. 주요 기능으로는 도형을 그룹에 추가하는 add_shape 함수, 그룹에서 도형을 제거하는 remove_shape 함수, 그룹이 선택되었는지 확인하는 select_Group 함수, 그리고 그룹의 영역을 그리는 selectRect 함수를 포함했습니다. 또한, 각 도형의 영역을 알 수 있도록 returnX 와 returnY 함수도 정의했습니다.

ChildView 클래스에는 여러 그룹을 관리할 수 있도록 GroupShape 클래스의 벡터를 추가했습니다. 이벤트가 발생할 때 그룹을 생성하고 선택된 도형들을 해당 그룹에 추가하도록 구현했습니다. 하지만 도형의 영역과 그룹의 영역이 중복되어 그려지는 문제가 발생했습니다. 이를 해결하기 위해 CMyShape 클래스에 도형이 어떤 그룹에 속해 있는지 확인하는 int 형 벡터 변수 gr 을 추가했습니다. 그룹이 생성될 때마다 도형의 gr 변수에 그룹 번호를 추가하여 그룹 개수와 일치시키고, add_shape 함수에서 도형의 gr 벡터 맨 끝 값을 1로 설정하여 그룹에 속해 있음을 표시했습니다.도형이 그룹에 속해 있는지 확인하기 위해 CMyShape 클래스에 Grouped 함수를 추가했습니다. 이 함수는 gr 벡터에 1이 존재하면 true, 그렇지 않으면 false를 반환합니다. true를 반환하는 도형은 그룹에 속해 있으므로 영역을 그리지 않도록 했습니다.그룹 안에 그룹이 속해 있는 경우를 처리하기 위해 GroupShape 클래스에 bool 형 변수 gr 을 추가하고, 그룹이 그룹화되었는지 확인하는 Grouped 함수를 만들었습니다. 이 함수는 멤버 변수로 있는 도형들이 자신보다 나중에 생성된 그룹에 속해 있으면 gr 변수를 true로, 그렇지 않으면 false로 설정합니다. gr 변수가 false이면 영역을 그리고, true이면 그리지 않도록 하여 그룹 내 그룹 문제를 해결했습니다.그룹 해제(Ungroup)는 remove_shape 함수를 통해 이루어집니다. 그룹 해제 시 선택된 그룹의 인덱스 정보를 remove_shape 함수에 전달하여 그룹에서 도형을 삭제할 때 해당 도형의 gr 벡터의 인덱스 값을 0으로 변경하여 더 이상 해당 그룹에 속하지 않음을 표시하고, 도형의 그룹화 정보를 업데이트했습니다.

9.

CMyShape*를 저장할 벡터를 만들고, 선택된 도형들을 처리하는 과정을 단계별로 해결해 나가면서 자신감을 얻을 수 있었습니다.특히, 선택된 도형들을 앞으로 또는 뒤로 정렬할 때, 벡터의 순서를 유지하거나 반대로 하여 삽입하는 방식은 매우 효율적이라고 느꼈습니다.

선택된 도형을 앞으로 또는 뒤로 정렬하기 위해 CMyShape*를 저장할 수 있는 벡터를 만들었습니다. 이 벡터에는 선택된 도형들이 순서대로 담기며, 이는 선택된 도형들의 정렬 상태를 유지하기 위함입니다. 선택된 도형들을 CChildView 클래스의 도형을 담는 멤버 변수에서 제거하여, 선택되지 않은 도형들만 남도록 했습니다. 그런 다음, 선택된 도형들을 앞으로 정렬할 때는 새로 만든 벡터의 도형들을 CChildView 멤버 변수의 뒤에 삽입하고, 뒤로 정렬할 때는 벡터에 담았던 순서를 반대로 하여 삽입함으로써 선택된 도형들의 순서가 변경되지 않도록 했습니다.

10.

Action 메뉴 아래에 여러 기능을 추가하면서, 사용자 인터페이스를 어떻게 구성해야 할지 깊이 생각하게 되었습니다. 툴바에 버튼을 추가하고 각각의 버튼이 메뉴와 동일한 이벤트를 처리하도록 설정하는 작업은 실제로 프로그램의

사용성을 크게 향상시켰습니다.CChildView 클래스에 도형의 개수를 표시하는 텍스트를 추가하면서, 프로그램이 실시간으로 상태를 반영하는 것을 보며 큰 만족감을 느꼈습니다.

메뉴에는 Action 메뉴 아래에 Draw, Select, Group, Ungroup, Align 메뉴를 추가했습니다. Draw 메뉴에는 Rectangle, Circle, Curve, Star 를 넣어 사용자가 원하는 도형을 선택할 수 있도록 했습니다. Align 메뉴에는 Bring Front 와 Bring Back 을 추가하여 도형을 앞으로 또는 뒤로 정렬할 수 있게 했습니다. 툴바에는 사각형, 원, 커브, 별 모양의 버튼을 추가하여 메뉴의 Rectangle, Circle, Curve, Star 와 동일한 이벤트 처리를 하도록 했으며, 툴바의 S 버튼은 Select, G 버튼은 Group, U 버튼은 Ungroup 기능을 작동하도록 설정했습니다.CChildView 클래스에는 현재 그려진 도형들의 개수를 나타내는 텍스트와 선택된 도형들의 개수를 나타내는 텍스트를 표시했습니다. 이를 위해 CChildView 클래스에 numShape(총 도형 개수)와 numSelect(선택된 도형 개수)라는 멤버 변수를 추가했습니다. 그룹화된 도형과 그룹을 하나의 도형으로 취급하기 위해, numShape 는 그룹화되지 않은 도형과 그룹만을 세도록 했습니다. 선택된 도형의 개수를 세는 방식도 동일하게 그룹화되지 않은 선택된 도형과 그룹만을 포함하도록 했습니다.이 정보를 통해 numSelect 값이 2 개 이상일 때만 Group 기능이 활성화되도록 하고, 1 개 이상일 때만 Ungroup 기능이 활성화되도록 했습니다.

11.

WM_CONTEXTMENU 핸들러를 추가하여 사용자가 우클릭했을 때 메인 프레임의 메뉴 정보를 불러와, 우클릭한 위치에 Action 메뉴가 표시되도록 구현했습니다.

실제로 우클릭한 위치에 Action 메뉴가 정확히 표시되는 것을 확인했을 때는 큰 성취감을 느꼈습니다. 이러한 사용자 인터페이스 개선 작업은 프로그램을 더욱 직관적이고 사용하기 쉽게 만들어 준다는 것을 깨달았습니다.